

Classifying Butterfly & Moth Species Using Convolutional Neural Networks

By: Durge Kumar, Katie Luce, Richa Tiwari, Jesus Daniel Martinez

CNN + Problem Motivation (Application Context)

What is Convolutional Neural Networks:

A convolutional neural network (CNN) is a type of deep learning model that uses multi layered neural networks with weighted nodes and thresholds to automatically process three dimensional image data and learn patterns for image classification and object recognition tasks.

Problem Motivation

Classifying insects from images is challenging due to visual similarity between species.

Goal: Build an automated system to classify images accurately using CNN model

Dataset: Target: Species of the Butterflies and Moths



Key Technology and Solutions

1) Why CNN?

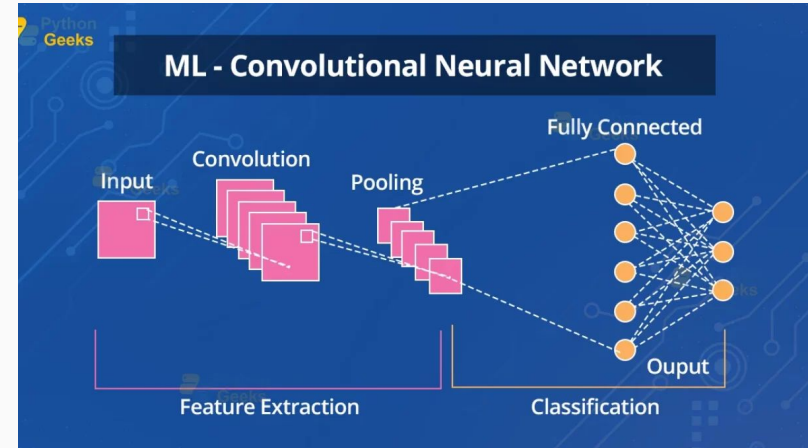
- Designed for image data (captures patterns like edges, shapes, textures)

2) Key components:

- Convolutional layers → extract features
- Pooling → reduce dimensionality
- Fully connected layers → classification

3) Pipeline

- Image resizing
- Label encoding
- Train/test split
- Model training



General Implementation Pathway

- 1) **Loading in Data + Importing Libraries**
- 2) **Exploratory Analysis**
 - a) Bar Chart: Number of pictures per species
- 3) **Model Building - 2 models, from-scratch and improved**
 - a) Cleaning Data + Preprocess Images(Resizing) + Encoding Labels
 - b) Train/Validation Test Split (80/20 split)
 - c) Build CNN Model
 - d) Compile/Train/Evaluate Model
- 4) **Evaluation (Visualization)**
 - a) Accuracy

Loading/Preparing Data + Exploratory Analysis



INDRA SWALLOW



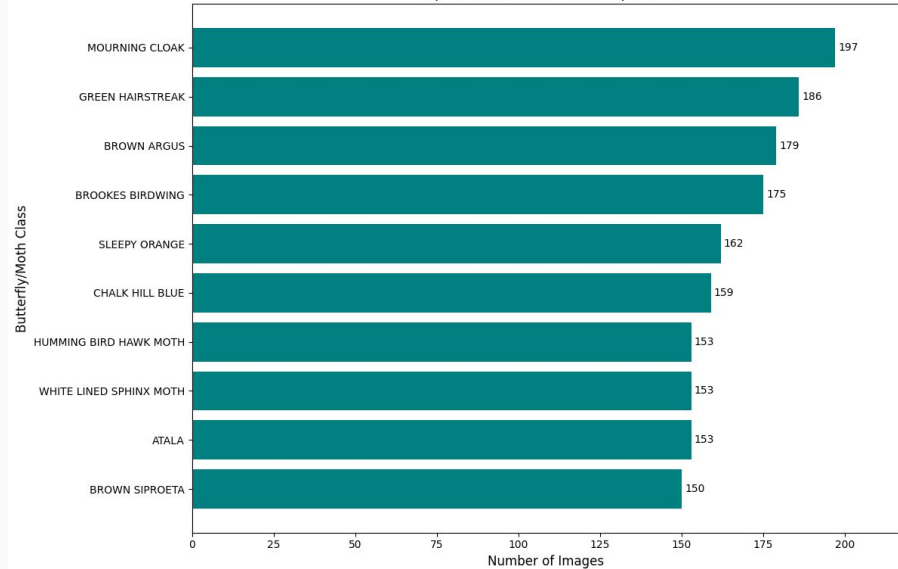
DANAID EGGFLY



RED POSTMAN



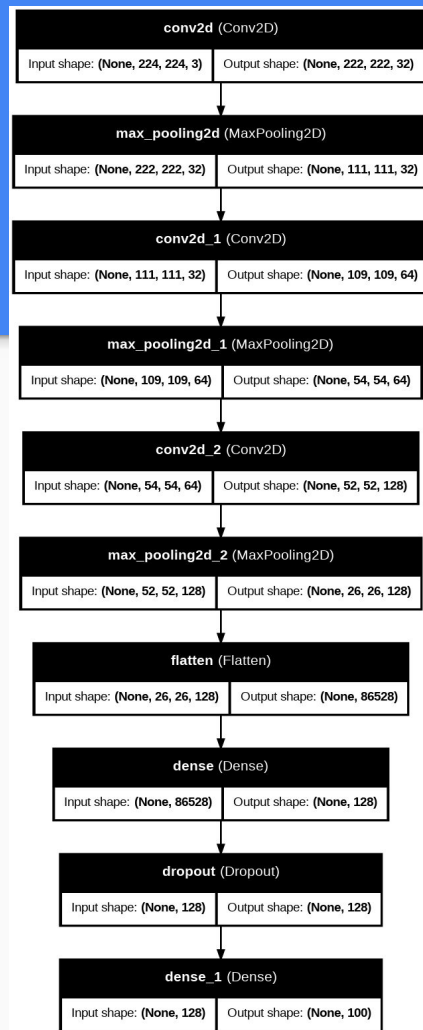
Top 10 Most Abundant Species



Model Building

The way our CNN model works is by extracting features with **convolutions**, shrinking them with pooling, and classifying them with dense layers.

- **Convolutional Layers (conv2d)**
- **Pooling Layers (max_pooling2d)**
- **Flattening (flatten)**
- **Dense**
- **Dropout**

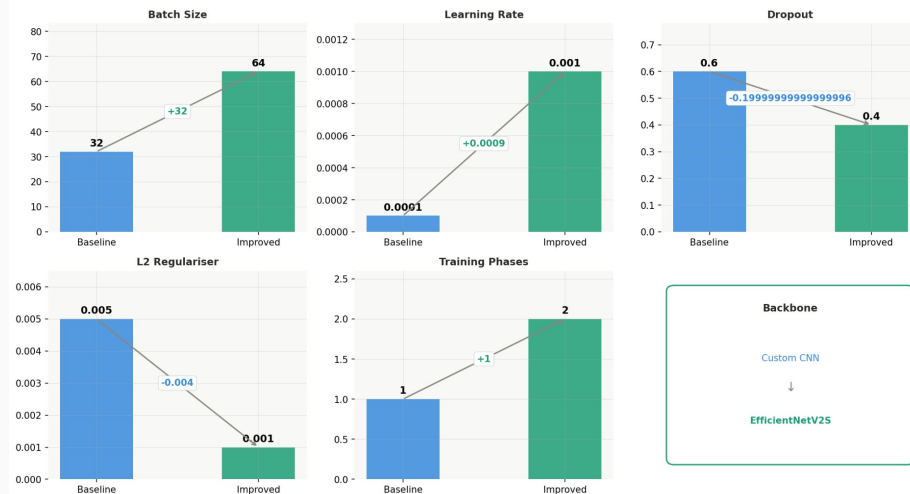


Tuning - Hyperparameters

Hyperparameter Changes: Baseline → Improved

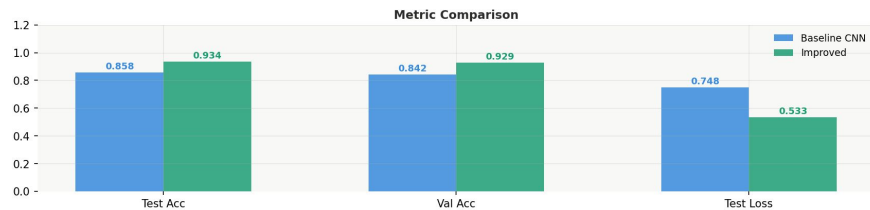
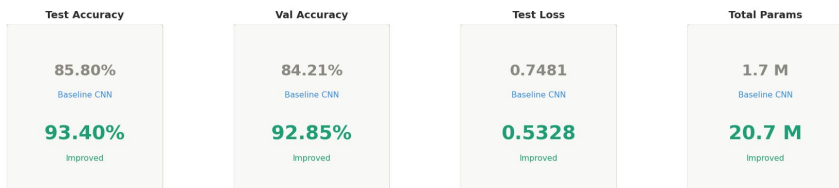
Hyperparameter	Baseline	Improved	Direction
Backbone	Custom CNN	EfficientNetV2S	—
Batch Size	32	64	▲
Learning Rate	0.0001	0.001	▲
Dropout	0.6	0.4	▼
L2 Regulariser	0.005	0.001	▼
Training Phases	1	2	▲
Augmentation	6 ops	7 ops (+Brightness)	—

Hyperparameter Changes: Baseline → Improved



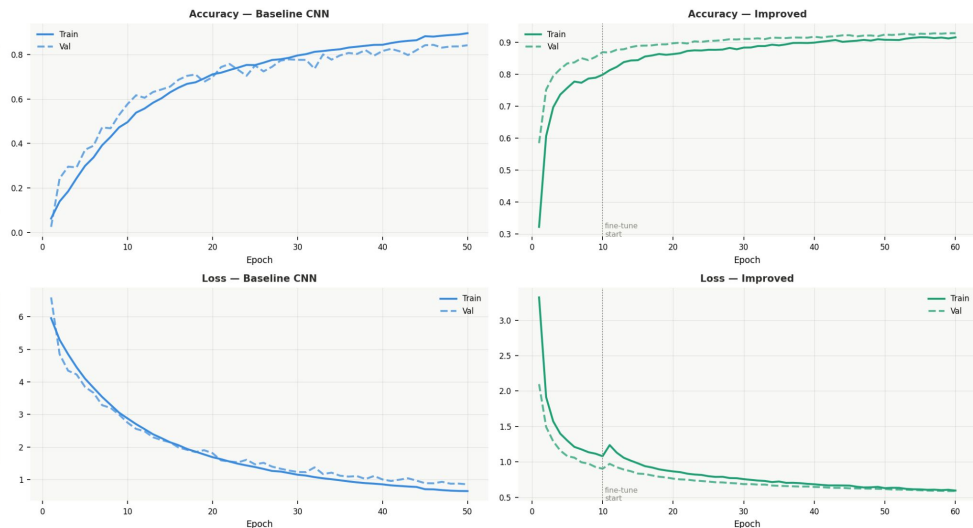
Evaluation

Results Summary



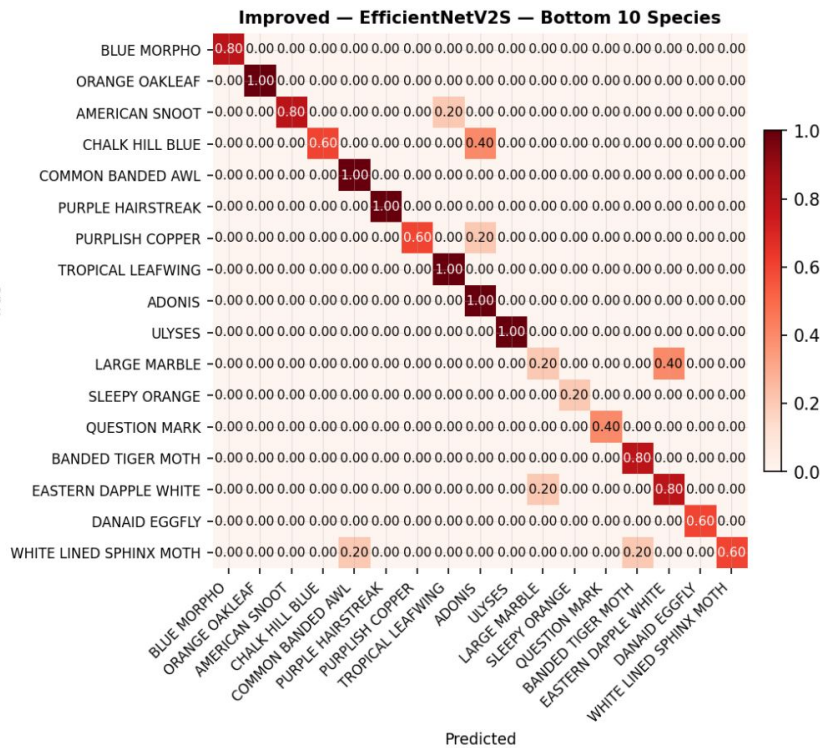
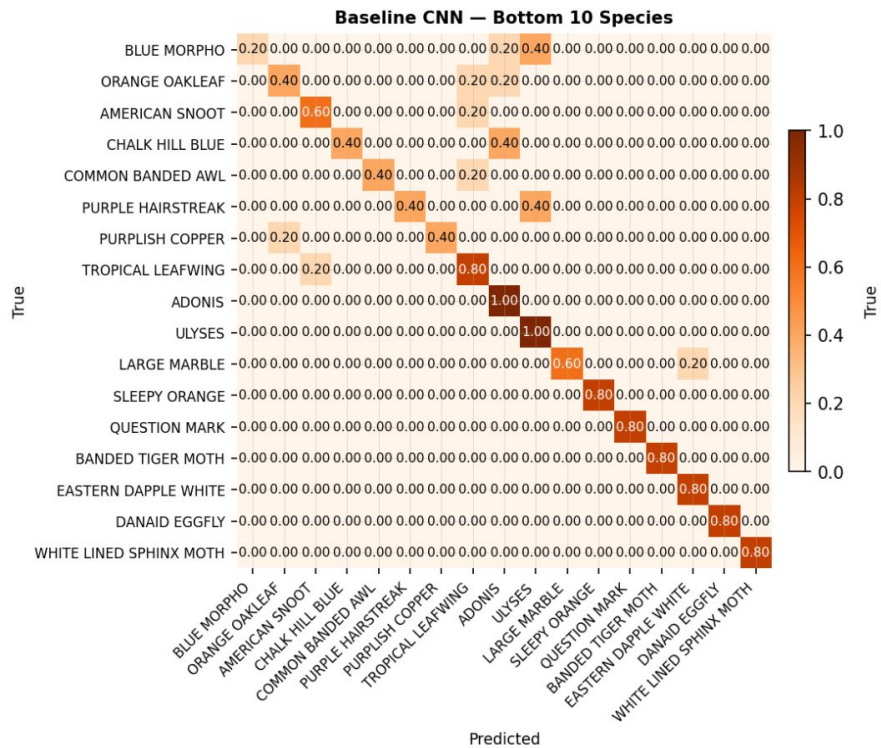
▲ Test accuracy improved by +7.60% (+7.60 pp)

Training Curves



Evaluation by Species

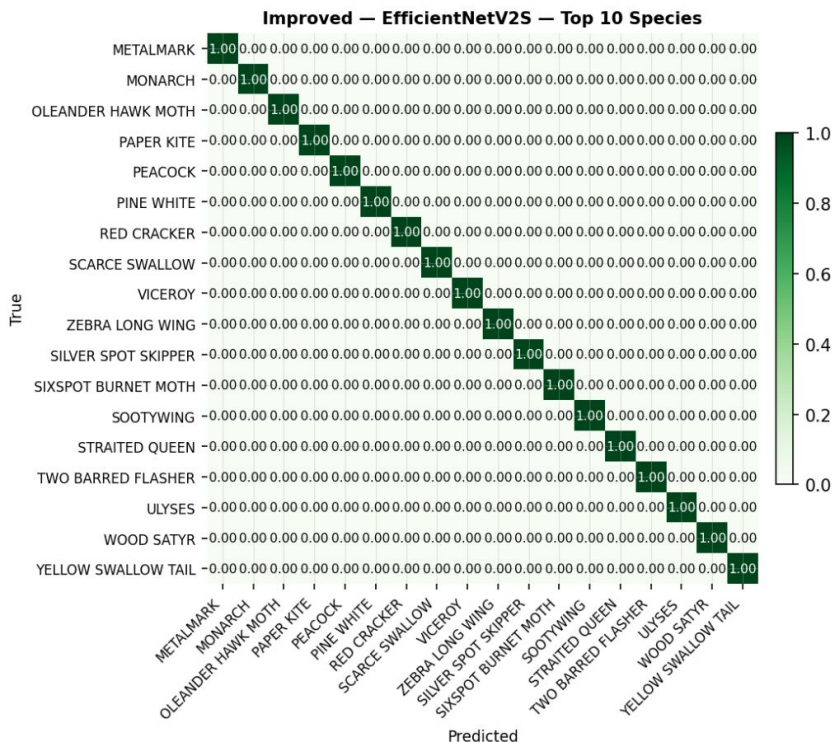
Confusion Matrices- Bottom Species by F1-Score (normalised)



Visual example



Confusion Matrices- Top Species by F1-Score (normalised)



QUICK CODE WALK THROUGH

Build Model

```
In [15]: # --- C3. Build model - EfficientNetV2S backbone (frozen for phase 1) ---
base_model = tf.keras.applications.EfficientNetV2S(
    include_top=False, weights='imagenet', input_shape=(224, 224, 3)
)
base_model.trainable = False

inputs = tf.keras.Input(shape=(224, 224, 3))
x = aug_b(inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256, activation='relu',
    kernel_regularizer=tf.keras.regularizers.l2(0.001))(x)
x = layers.Dropout(0.4)(x)
outputs = layers.Dense(num_classes, activation='softmax')(x)

model_b = tf.keras.Model(inputs, outputs)
model_b.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/efficientnet_v2/efficientnetv2-s_notop.h5
82420632/82420632 6s 0us/step

Model: "functional_3"

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 224, 224, 3)	0
augmentation_improved (Sequential)	(None, 224, 224, 3)	0
efficientnetv2-s (Functional)	(None, 7, 7, 1280)	20,331,360
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dense_2 (Dense)	(None, 256)	327,936
dropout_1 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 1000)	25,700

QUICK CODE WALK THROUGH

CNN from scratch

```
model = models.Sequential([
    data_augmentation,

    # Block 1
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),

    # Block 2
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),

    # Block 3
    layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),

    # Block 4
    layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),

    # Block 5
    layers.Conv2D(512, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),

    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(0.005)),
    layers.Dropout(0.6),
    layers.Dense(num_classes, activation='softmax')
])

model.build(input_shape=(None, 224, 224, 3))
model.summary()
```

CNN Improved(pre-trained)

```
# CHANGED: Load pretrained EfficientNetV2S backbone (frozen)
base_model = tf.keras.applications.EfficientNetV2S(
    include_top=False,
    weights='imagenet',
    input_shape=(224, 224, 3)
)
base_model.trainable = False # Freeze during phase 1

inputs = tf.keras.Input(shape=(224, 224, 3))
x = data_augmentation(inputs)
x = base_model(x, training=False) # training=False keeps BN frozen
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(
    256, activation='relu',
    kernel_regularizer=tf.keras.regularizers.l2(0.001) # CHANGED: 0.005 → 0.001 (less aggressive)
)(x)
x = layers.Dropout(0.4)(x) # CHANGED: 0.6 → 0.4 (pretrained features need less dropout)
outputs = layers.Dense(num_classes, activation='softmax')(x)

model = tf.keras.Model(inputs, outputs)
model.summary()
```


QUICK CODE WALK THROUGH

Training

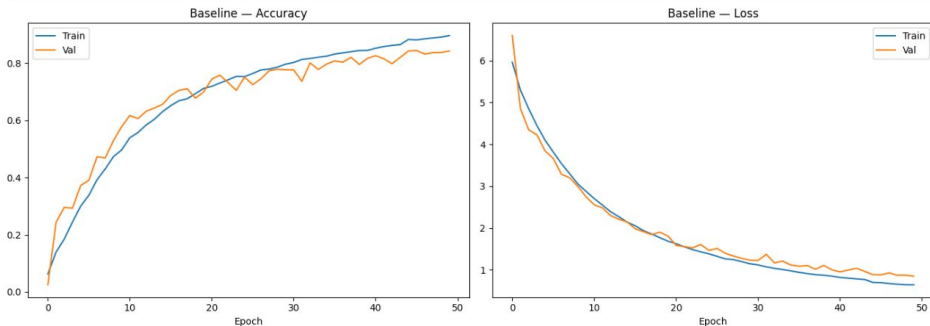
```
In [9]: # -- B4. Compile -----  
model_a.compile(  
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
callbacks_a = [  
    EarlyStopping(monitor='val_accuracy', patience=7,  
                  restore_best_weights=True, verbose=1),  
    ReduceLROnPlateau(monitor='val_loss', factor=0.5,  
                     patience=3, min_lr=1e-7, verbose=1)  
]
```

```
In [10]: # -- B5. Train -----  
print('Training Baseline CNN...')  
history_a = model_a.fit(  
    train_ds_a,  
    validation_data=val_ds_a,  
    epochs=50,  
    callbacks=callbacks_a  
)
```

Training Baseline CNN...

```
Epoch 1/50  
315/315 ----- 753s 540ms/step - accuracy: 0.0627 - loss: 5.9579 - val_accuracy: 0.0251 - val_loss: 6.5954 - learning_rate: 1.00  
00e-04  
Epoch 2/50  
315/315 ----- 10s 33ms/step - accuracy: 0.1394 - loss: 5.2922 - val_accuracy: 0.2430 - val_loss: 4.8509 - learning_rate: 1.0000  
e-04  
Epoch 3/50  
315/315 ----- 10s 33ms/step - accuracy: 0.1848 - loss: 4.8502 - val_accuracy: 0.2959 - val_loss: 4.3441 - learning_rate: 1.0000  
e-04
```

```
In [12]: # -- B7. Quick training plot -----  
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))  
ax1.plot(history_a.history['accuracy'], label='Train')  
ax1.plot(history_a.history['val_accuracy'], label='Val')  
ax1.set_title('Baseline - Accuracy'); ax1.set_xlabel('Epoch'); ax1.legend()  
ax2.plot(history_a.history['loss'], label='Train')  
ax2.plot(history_a.history['val_loss'], label='Val')  
ax2.set_title('Baseline - Loss'); ax2.set_xlabel('Epoch'); ax2.legend()  
plt.tight_layout()  
plt.savefig(base_path + 'training_history_baseline.png', dpi=150)  
plt.show()
```



Citations

Argue, C. L. (2012). The pollination biology of North American orchids: Volume 1: North of Florida and Mexico. Springer. <https://doi.org/10.1007/978-1-4614-0592-4>

Borkowsky, C., & Westwood, A. (2009). Seed capsule production in the endangered western prairie fringed orchid (*Platanthera praeclara*) in relation to sphinx moth (Lepidoptera: Sphingidae) activity. *Journal of the Lepidopterists' Society*, 63(2), 110-117. [https://images.peabody.yale.edu/lepsoc/jls/2000s/2009-63-2-110.pdf](https://images.peabody.yale.edu/lepsoc/jls/2000s/2009/2009-63-2-110.pdf)

Dettmers, T. (2015, March 26). Understanding convolution in deep learning. <https://timdettmers.com/2015/03/26/convolution-deep-learning/>

GeeksforGeeks. (2026, April 14). Introduction to convolution neural network. <https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/>

Hahn, M., & Brühl, C. A. (2016). The secret pollinators: An overview of moth pollination with a focus on Europe and North America. *Arthropod-Plant Interactions*, 10(1), 21–28. <https://doi.org/10.1007/s11829-016-9414-3>

Kasinathan, T., Singaraju, D., & Uyyala, S. R. (2021). Insect classification and detection in field crops using modern machine learning techniques. *Information Processing in Agriculture*, 8(3), 446–457. <https://doi.org/10.1016/j.inpa.2020.09.006>

Lim, S., Kim, S., Park, S., & Kim, D. (2018). Development of application for forest insect classification using CNN. In 2018 15th International Conference on Control, Automation, Robotics and Vision (ICARCV) (pp. 1128–1131). IEEE. <https://doi.org/10.1109/ICARCV.2018.8581103>

NVIDIA. (2026). Convolutional neural network (CNN). NVIDIA Developer. <https://developer.nvidia.com/discover/convolutional-neural-network>

Ozdemir, D., & Kunduraci, M. S. (2022). Comparison of deep learning techniques for classification of the insects in order level with mobile software application. *IEEE Access*, 10, 35675–35684. <https://doi.org/10.1109/ACCESS.2022.3163380>

Soni, P. (2025, September 2). The Ultimate Guide to Deep Learning Hyperparameter Tuning - Train in Data's Blog. <https://www.blog.trainindata.com/the-ultimate-guide-to-deep-learning-hyperparameter-tuning/>

Spiesman, B. J., Gratton, C., Hatfield, R. G., Hsu, W. H., Jepsen, S., McCornack, B., Patel, K., & Wang, G. (2021). Assessing the potential for deep learning and computer vision to identify bumble bee species from images. *Scientific Reports*, 11(1), 7580. <https://doi.org/10.1038/s41598-021-87210-1>

Stark, T., Wurm, M., Ștefan, V., Wolf, F., Taubenböck, H., & Knight, T. M. (2025). Utilizing CNNs for classification and uncertainty quantification for 15 families of European fly pollinators. *PLOS ONE*, 20(9), e0323984. <https://doi.org/10.1371/journal.pone.0323984>

Ștefan, V., Stark, T., Wurm, M., Taubenböck, H., & Knight, T. M. (2025). Successes and limitations of pretrained YOLO detectors applied to unseen time-lapse images for automated pollinator monitoring. *Scientific Reports*, 15(1), 30671. <https://doi.org/10.1038/s41598-025-16140-z>

TensorFlow. (2024, June 7). `tf.keras.layers.Dropout`. https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dropout

TutorialPedia.org. (2025, November 29). Why validation accuracy exceeds training accuracy in TensorFlow/Keras: Deep learning for income prediction with dating site data. <https://www.tutorialpedia.org/blog/higher-validation-accuracy-than-training-accuracy-using-tensorflow-and-keras/>

Voronova, A. (2022, July 16). Neural network identifies insects, outperforming humans. Hackaday. <https://hackaday.com/2022/07/16/neural-network-identifies-insects-outperforming-humans/>

Westwood, A. R., & Borkowsky, C. L. (2004). Sphinx moth pollinators for the endangered western prairie fringed orchid, *Platanthera praeclara*, in Manitoba, Canada. *Journal of the Lepidopterists' Society*, 58(1), 13-20. <https://images.peabody.yale.edu/lepsoc/jls/2000s/2004/2004-58%281%2913-Westwood.pdf>

Willmer, P. (2011). *Pollination and floral ecology*. Princeton University Press. <https://doi.org/10.1515/9781400838943>

Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2104.00298>

Any Questions?